Item 2: Relational Statics

2.1. Statics of the Relational Data Model

- **Domains, Attributes and Relations**
- **Relational Schema representation**
- **Characteristics of a relation**
- **Inherent restrictions**
- **Semantic restrictions**

2.2. *Standard Query Language* for Describing (DDL)

Item 2.2: Relational Statics, elements

The relational model is based on the mathematical notion of *relation*:

Example: R: N x N , R $\{(x, y) / y = x^2\}$

R {(1, 1), (2, 4), (3, 9), ...}

Domain: set of values of the same nature

Relation: subset of the Cartesian product of n domains

person: DNI x Name

person: {(03456789, José), (03451940, Jack), ...}

Attribute: aspect shared by the elements of a relation defined on a domain (and which valid values are bounded to it).

Example: *NIA_student* is defined on the NIA domain (nine digits natural numbers: $\geq 9 \cdot 10^8$, $<10^9$) and is a feature of the student relation

Item 2.2: Relational Statics, notations

Relation:

Set of occurrences representing and defining a collection of individuals in the real world (more precisely, in the universe of discourse, which is the subset of the real world focused by each particular design case).

Definitions of a Relation:

- **Intensive Definition:**
  semantic invariable definition of a relation (schema)
- **Extensive Definition:**
  set of tuples of a relation in a given moment in time (t_i).

Notations: remember to use *camelCase* and **snake_case** for labels

- intensive: relational graph
- extensive: tabular representation (a column per attribute, a row per tuple).

Item 2.2: Relational Statics, characteristics

<u>Characteristics</u> (of any relation):

- The number of attributes is named (relation) ***degree***
- The number of tuples is named (relation) ***cardinality***

<u>Example</u>: Relation Schema: Person (Name, DNI, Telephone)

cardinality

Name	DNI	Telephone
Javi	34567890	555-123456
Carlos	34767890	555-987654

Relation

degree

Item 2.1: Relational Inherent Restrictions

The Relation is defined as a set of attributes

- Attribute order does not matter

The Relation is defined as a set of individuals

- Tuple order does not matter

Individuals are not repeated (they are all unique)

- There are no two identical tuples

Individuals are of the same nature

- They share definition (all tuples have the same arity)
- Attributes are common, single-valued, atomic characteristics
  - Single-valued: each attribute takes at most one domain value per tuple
  - Atomic: non-divisible for the aim they are set (e.g., *postal_address*).

Item 2.1: Integrity Inherent Restrictions

- ***Integrity of Entity***: *every existing row is identifiable*

  → no attribute from the primary key can take null value

  - null value: conventional value used for representing either unknown information or inapplicable attribute.

  - Attributes where null is allowed, are noted with an asterisk *

- ***Referential Integrity***: *anything referenced always exists*

  Foreign Key: attribute (or set of attributes) from the *referencing* relation which is linked to another attribute (or as many attributes) within another relation (*referenced relation*), so its valuation is restricted to

  - existent values in the referenced key (attribute/s)
  - null value

  } ***Referential Integrity***

Item 2.1: Optional Foreign Keys

- When the foreign key is null, integrity check is skipped (*orphan*)
- When the referenced superkey comprises several attributes, the foreign key should include as many. If any attribute in the foreign key is optional, the integrity can be defined in several ways:

Integrity Types: (*for multi-attribute optional FK*)

- Complete (*match full*): *all FK attributes work as one*
  → all attributes take null value, or none do.
- Partial (*match partial*): non-null part must be checked
  → there is at least one 'potential father row' with those values
- **Weak** (*match simple*): if there is any null value, **no check**
  → default behavior in Oracle DB

Item 2.1: Semantic Restrictions

Semantic

Choosing a primary key: noted PRIMARY KEY

Marking alternative keys (candidate/secondary): UNIQUE

Fixing obligatoriness: noted NOT NULL

Identifying associations: noted FOREIGN KEY

Choosing Integrity Option: automatic action to be executed when the referenced value disappears (*father row* is either deleted or updated).

General Rejection Semantic Restrictions

Item 2.1: Referential Integrity Options

They are five options (rules). They define the automatic action to be run when any operation (Delete/Update) endanger the Referential Integrity.

- **Restrict**: DR / UR

  Any operation on fathers with child is prevented (checks before execution).

- **No Action** : DNA / UNA

  Execution is prevented when breaks referential integrity (checks after).

- **Cascade** (propagation): DC / UC

  Operation will be extended to the tuples containing old non-valid values

- **Set Null**: DSN / USN

  Old (non valid) foreign key values will be substituted by null value

- **Set Default**: DSD / USD

  Old (non valid) foreign key values will be substituted by default value

Item 2.1: Rejection Semantic Restrictions

Consist in checking a conditional expression before executing actualizations. If the operation to be performed on the DB (insertion, update or deletion) does not meet that condition, it is rejected.

There are two types: Simple and Assertion (general restriction)

Simple: It is applied on an only relational element (domain/relation/row)

Their implementation is part of the element implementation.

Syntax: **CHECK <conditional_expr>**

Example:

```
CREATE TABLE STUDENT(
  NIA CHAR(9),
  MARK NUMBER(2,0), ...,
  CONSTRAINT ck_mark CHECK (mark BETWEEN 0 AND 10)
  );
```

Item 2.1: Rejection Semantic Restrictions

Assertion

It is a general restriction (imposed on the whole DB).

It should concern several relational elements (several tables, several rows, …).

It is itself an independent element (must have a name).

All the defined assertions are run after every actualization on the DB.

→ very high computational cost!

→ most commercial DBMS don't implement them!

Syntax: **CREATE ASSERTION <name> CHECK <conditnl_exp>;**

```
CREATE ASSERTION monthly_wage_DB_admin CHECK NOT EXIST
        (SELECT 'X' FROM EMPLOYEES A JOIN DEPARTMENT B
                ON (A.dep=B.cod_dep)
                WHERE B.position='DB Admin' and A.wage<8000);
```

Item 2.1: Domains & Datatypes

- The concept of *domain* is useful to complete the definition of the model, but it tends to remain theoretical.
- Most implementations rely on the concept of data type:
  set of values of the same nature, and with a defined operability.
- Data types can be classified by their:
  - Nature: *Numeric*, *Alphanumeric*, *Dates* and *Binary*
  - Implementation: *Native*; *Derived* (user def.); and *External* (pack.)
  - Operability: comparable, arithmetic, aggregable, …
- The concept of *'domain'* is similar to that of a **derived data type**, defined as a native datatype (restricted by implementation) with one, none, or several ***rejection semantic constraints***.

  → *because of this, most DBMS skip this object, relying on derived datatypes*

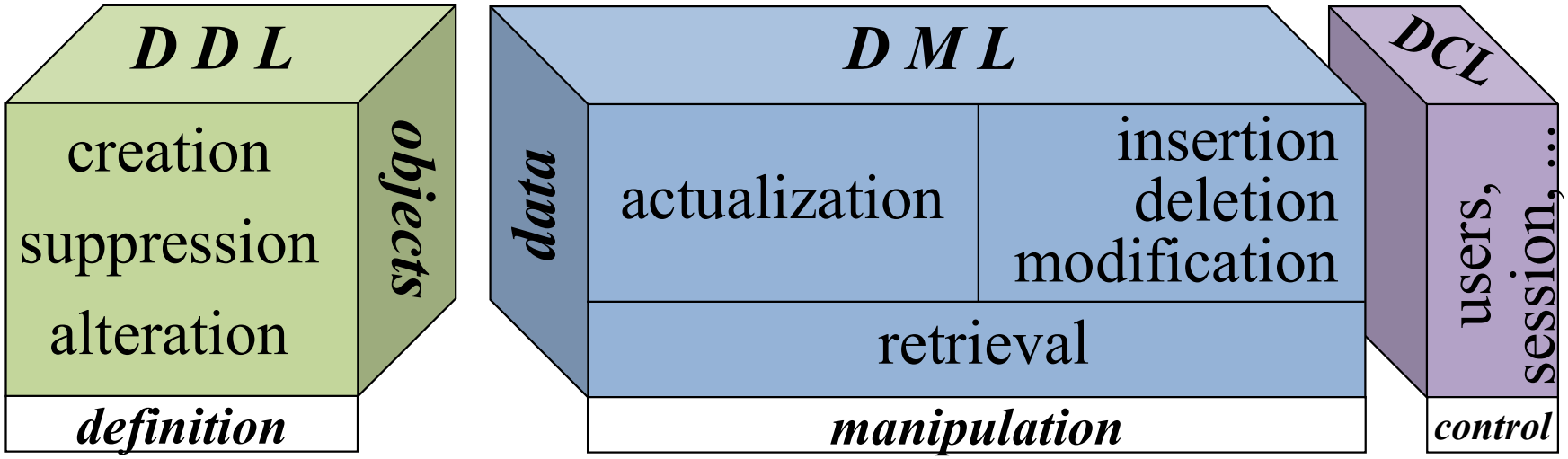
Item 2.1: Semantic Constraints

In the following examples, indicate which are semantic rejection constraints and, in case, suggest their typology.

- *the 'age' attribute has a value less than or equal to 120*
- *the 'city' attribute is registered as valid in the table 'Cities'*
- *the 'shift' can be "morning", "afternoon" or "night"*
- *the 'age' attribute has a value between 18 and 67*
- *the 'alive' attribute admits the values "YES" and "NO"*
- *there are no more than 40 students per course (same value in 'group')*
- *if there is a 'death_date', it is later than the 'birth_date'*
- *maximum length of Name is 25, that of Surname 20, and that of both of them jointly is 40 characters long.*

Item 2.2: Structured Query Language

- Developed by D.Chamberlin and R.Boyce (IBM, '70s)
- There are several versions of "standard", and many variants
- Most widespread language for handling (structured) DBs
- Instructions of the SQL Language: three subsets
  - **Data Definition Language**
  - **Data Manipulation Language**
  - **Data Control Language**



Item 2.2: SQL – Data Definition Lang

- Three basic actions:
  - Creation (`CREATE`)
  - Suppression (`DROP`)
  - Alteration (`ALTER`)

- On:
  - Schemas
  - Tables
  - Views
  - Domains
  - Sequences…

Item 2.2: SQL – Creating a TABLE

```
CREATE TABLE table_name (
   table_element  [, table_element [,…] ]   )
  ;
```

```
table_element := { column_def | table_constraint }

column_def := column_name  datatype(max_size)
              [column_val]
              [column_constraint]

Column_val :={ DEFAULT value
            | GENERATED {ALWAYS|BY DEFAULT} AS IDENTITY
              [(START WITH int [,INCREMENT BY int])]  }
```

Item 2.2: SQL Datatypes

- They can be classified by their:
  - Implementation: Native, User and External (package, etc.)
  - Nature: Numeric, Alphanumeric, Dates and Binary
  - Operability: comparable, aggregable, …

DATATYPES	Description	En Oracle
Numeric	Natural number (fixed point) real number (float) real number (5 bytes) (float) real number (9 bytes)	**NUMBER(n)** NUMBER(n,p) BINARY_FLOAT BINARY_DOUBLE
Alphanumeric	(fixed length) character string (variable length) character string (variable length) Unicode char string	CHAR(n) **VARCHAR2(n)** NVARCHAR2(n)
Time	date and time (7 bytes) Date, time and fraction of sec. (11 bytes)	**DATE** TIMESTAMP
Binary	'raw' binary, (undefined length) text , … large objects …	RAW / LONG / LONG RAW … BLOB / CLOB / NCLOB / BFILE

Item 2.2: SQL Constraints

RESTRICTIONS:

- column_constraint
  [CONSTRAINT name]

```
[NOT] NULL
PRIMARY KEY
UNIQUE
REFERENCES…
CHECK (… )
```

[state]

```
ENABLE
DISABLE
...
```

- **table_constraint**
  [CONSTRAINT name]

```
PRIMARY KEY (col[,…])
UNIQUE (col[,…])
FOREIGN KEY (col[,…]) REFERENCES …
CHECK (… )
```

[state]

Item 2.2: SQL – Linking Tables

Foreign key: is another semantic constraint (at columns or table_line)

```
FOREIGN KEY (<columns>)
        REFERENCES <table_name>
        [(<columns>)]
              [ON DELETE <action>]
        [ON UPDATE <action>]
```

Referential actions (Integrity options)

CASCADE | SET NULL | RESTRICT | NO ACTION |SET DEFAULT

- Most RDBMS only support a subset of those integrity options
- For example, Oracle DB (r. 12c) supports the following ones:
  - On Update: only observes **NO ACTION** (by default, cannot be stated)
  - On Delete: by default **NO ACTION** ; it also observes **CASCADE** **SET NULL**

Item 2.2: SQL – Create Table Example

```
DROP TABLE course;

CREATE TABLE Course (
   subject  VARCHAR2(100),
   degree   VARCHAR2(50),
   credits  NUMBER(2) NOT NULL,
   lecturer VARCHAR2(50),
   CONSTRAINT pk_course PRIMARY KEY(subject, degree),
   CONSTRAINT valid_ttl CHECK (credits>0),
   CONSTRAINT fk_course_degree FOREIGN KEY(degree)
       REFERENCES degree(name)
);
-- SQL is not case sensitive, remember to use snake_case

DESC course
```

Item 2.2: SQL – Relational Catalog

- It's a set of metadata-tables for recording all definitions (all objects)
- In Oracle it is named 'Data Dictionary', and here is a small sample of it:

```
/* Metadata of our tables can be found in USER_TABLES */
SELECT table_name FROM USER_TABLES;
DESC user_tables

/* Our tables' columns are described in USER_TAB_COLUMNS */
SELECT column_name, data_type, data_length, nullable
       FROM USER_TAB_COLUMNS
       WHERE table_name='course';

/* …and constraints are in USER_CONSTRAINTS */
SELECT CONSTRAINT_NAME, CONSTRAINT_TYPE
       FROM USER_CONSTRAINTS
       WHERE table_name='referencing';

/* take a further look at the foreign key constraint */
SELECT CONSTRAINT_NAME, R_CONSTRAINT_NAME, DELETE_RULE
       FROM USER_CONSTRAINTS
       WHERE table_name='course' AND CONSTRAINT_TYPE='R';
```

C	Check
P	Primary K
U	Unique
R	Referential

Item 2.2: SQL – Other DDL sentences

2. **Removal**:

```
DROP <element> <name>;
```

(just getting rid of data):

```
TRUNCATE <element> <name>;
```

3. **Amendment**:

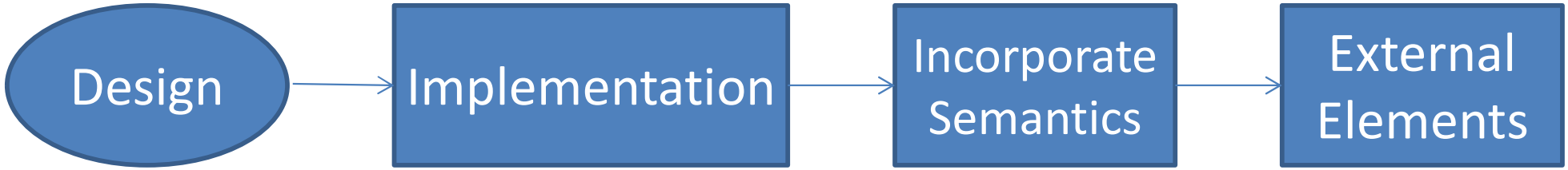
```
ALTER <element> <name>
      {ADD|ALTER|DROP} <element>
      [<definition>];
```

EXAMPLES:

```
ALTER TABLE example ADD (age NUMBER(2));

ALTER TABLE example ADD CONSTRAINT max_age CHECK (age<100);

SELECT CONSTRAINT_NAME, SEARCH_CONDITION
       FROM USER_CONSTRAINTS
       WHERE table_name='example' AND CONSTRAINT_TYPE='C';

DROP TABLE example;
```

Item 2.2: Relational Design

- Every association has to be accompanied with the chosen integrity rule for each action endangering integrity
- The set of all the relation definitions (schemata) of a RDB, along with their associations and the RI options *(on **D**elete / on **U**pdate)* chosen is known as **Relational Schema.**
- The diagram we use for describing this is known as Relational Graph.
- This bare design must be completed by adding documentation detailing semantics (labels, added constraints, missing constraints). It will serve for later semantic completion (and/or specifications refinement).



Item 2.2: Design Documentation

- An *explicit semantic assumption* is any piece of meta-information provided by the client/user (*requirement*)

- Any ***non-observed explicit semantic assumption*** (that cannot be included in the proposed design) must be identified and explained (optionally, you can include a further solution for the deficiency).

- On the other hand, the design usually require more information than the explicitly provided by the client/user.
  Implicit semantic assumptions are pieces of meta-information included in the design yet absent in the requirements provided, and which observance will constrain the semantic coverage (data definition is restricted).

Item 2.2: Implicit Semantic Assumptions

Example: model clients with name and surname/s (up to two), identified by DNI, who own car/s (at least one) with number plate, make, model, and color.

Clients (DNI, Name, Surname1, Surname2*)

DC / UC

Car (NumberPlate, Make, Model, Color, Owner*)

Wrong - Right

- There cannot be two clients with the same 'DNI'
- Cars are identified by their 'number plate'
- Second surname (of the client) is optional
- Every car has an 'owner'
- If a client is removed, his/her cars must be removed as well.
- All cars have 'make' and 'model' (obliged)
- The 'DNI' are alphanumeric strings of nine characters

Item 2.2: Non-Observed Explicit Semantics

Example: client with name and surname/s, identified by DNI, who own car/s (at least one) of some make, model, and color.

Clients (DNI, Name, Surname1, Surname2*)

DC / UC

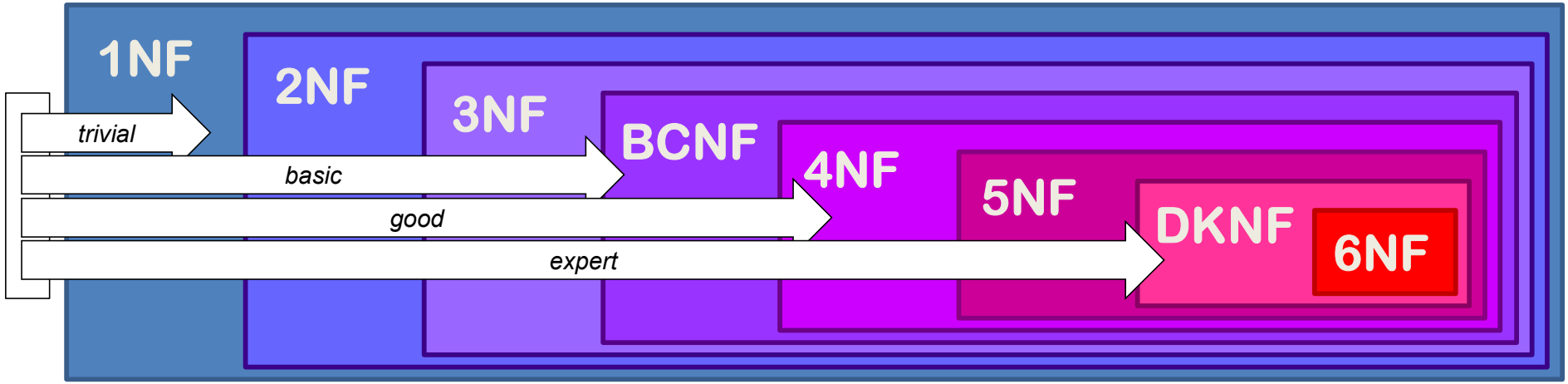
Cars (NumberPlate, Make, Model, Color, Owner*)

Wrong - Right

- You cannot record several cars for an only client.
- It is not observed that a car has several 'owners'.
- It is not assured that every car has an 'owner'.
- It is not assured that every client has at least one car.
- It is not assured that the DNI values are not repeated (as *clients*)
- It is not observed that the 'DNI' is an eight-digit natural number
- 'Models' are not validated with the cars 'make'.

Item 2.3: The Normal Forms

- **Normalization** is the process of checking and transforming a relational design to avoid logical redundancies and further inconsistencies, thus ensuring data integrity.
- The *normal forms* are progressive restrictions set on a relational design for locating such redundancies (design flaws) and supporting their amendment.
- These forms sequence is: 1NF, 2NF, 3NF, BCNF, 4NF, 5NF, DKNF, and 6NF while it should be noted that most advanced are only required in special cases (for most designs, it is enough to check 4NF, and even the BCNF)
- The first four were proposed by E.F. Codd (the fourth, along with R.F.Boyce). The other were proposed by R. Fagin (4NF, 5NF and DKNF), and C.J.Date (6NF).



Item 2.3: Functional Dependency $x \rightarrow y$

- any attribute ***y*** (*dependent*) is functionally dependent by ***x*** (*determinant*) iff its valuation is determined by the other's value (*x*). Example: DNI → name

- Properties (Armstrong axioms): $\forall$ x,y,z attributes from R relation
  - Reflexive: $\{x,y\} \rightarrow x$
  - Augmentative: if $x \rightarrow y$, then $\{x,z\} \rightarrow \{y,z\}$
  - Transitive: if $x \rightarrow y \wedge y \rightarrow z$, then $x \rightarrow z$

- Derived rules:
  - Union: if $x \rightarrow y \wedge x \rightarrow z$, then $x \rightarrow \{y,z\}$
  - Decomposition: if $x \rightarrow \{y,z\}$, then $x \rightarrow y \wedge x \rightarrow z$
  - Composition: if $x \rightarrow z \wedge y \rightarrow w$, then $\{x,y\} \rightarrow \{z,w\}$
  - Pseudo-transitivity: if $x \rightarrow y \wedge yz \rightarrow w$, then $\{x,z\} \rightarrow w$

Item 2.3: Functional Dependency $x \rightarrow y$

- Re-definition of the **superkey** concept:
  - x is a superkey iff $\forall$ y $\in$ R meets $x \rightarrow y$

- Redefining the concept of **candidate key**:
  - x is a candidate key iff
    - ✓ x is a superkey
    - ✓ $-\exists$ $z \subset x$ superkey.

- **<u>Prime attributes</u>**: those that belong to any candidate key.

- **<u>Heath's Theorem</u>** (*lossless decomposition*):

  any relation $R \supset \{x,y,z\}$, meeting $x \rightarrow y$
  can be split into $\pi_{xy}(R)$ and $\pi_{xz}(R)$
  without loss of information.

Item 2.3: Basic Normal Forms

1NF R is in the first normal form (1NF) iff

- ✓ Its attributes are atomic (non-divisible) <u>for the aim they are set</u>.
- ✓ Its attributes are single-valued.
- ✓ It has a PK (not null minimum superkey)
- ✓ Its arity (degree) is constant.
- ✓ The order of both rows and columns is not important.

2NF R is in 2NF iff

- ✓ It is in 1FN and there are no *partial dependencies* from a candidate key

  -$\exists$ **x** $\rightarrow$**y**, $\forall$ **{x,y}** $\in$ **R**, **x** $\subset$ **CK**, ***y*** $\not\subset$ **CK** <u>(*x* is prime, *y* is not prime)</u>

<u>Example</u>: Orders(<u>id_client, date</u>, client_name, product)

The FD (id_cliente $\rightarrow$ client_name) breaks the 2NF

<u>Solution</u>: divide the relation R into two (applying Heath's theorem)

Item 2.3: Basic Normal Forms

3NF R is in 3NF iff

- ✓ It is in 2FN and there are no transitive dependencies from PK to non-primes.
  $\forall\ x \rightarrow y$, meets (x is superkey) $\vee$ ($y \subset x$) $\vee$ ($z \in \{y\text{-}x\}$, z is prime attribute)
  Example: branch(id_branch, town, province),, with *town*$\rightarrow$ *province*
  Not in 3NF (town is not superkey and province is non-prime attribute)

BCNF (Boyce-Codd): R is in BCNF iff

- ✓ It is in 2FN and there are no transitive dependencies
  $\forall\ x \rightarrow y$, meets (x is superkey) $\vee$ ($y \subset x$)

- **Not every 3NF relation is in FNBC:**
  Example: shows(room, session, movie)
  with {*room,session*}$\rightarrow$ movie , {*session,movie*}$\rightarrow$ *room*, and *movie* $\rightarrow$ *room*
  - all attributes are prime, so it is in 3FN
  - but it is not in FNBC, because of *movie* $\rightarrow$ *room*
- To fix this, we should apply the lossless decomposition on that FD.

Advanced Normal Forms: 4NF

<u>Multi-valued Functional Dependency</u> (MFD): **x→→ y**

x does not determine a sole value for *y*, but a value in a subset of the domain

<u>Example</u>: *uniforms (<u>sport, clothing, fieldtype</u>)*

{ (football, t-shirt, pitch), (football, t-shirt, court), (football, shorts, pitch), (football, shorts, court), (rugby, t-shirt, pitch), (rugby, cap, pitch) }

- this example meets *sport→→ clothing* (the value of *sport* constrains the value of the attribute *clothing*, regardless of the other attributes).
- provided that such restriction is met, if exists a tuple (football, boots, pitch) then there will be another tuple (football, boots, court)

<u>trivial MFD</u>: **x→→ y** is trivial when either $y \subset x$ or $y \cup x \equiv R$

4NF R is in 4FN iff it is in BCNF and

$\forall$ $x \rightarrow\rightarrow y$, either it is trivial DMF or *x* is superkey (there are no other DFM)

<u>Note</u>: BCFN is actually implicit, because → is a particular case of →→

Advanced Normal Forms: 5NF

Join Dependency (JD): $*(R_1, R_2, \ldots, R_n)$,
This restriction applies on a decomposition of a relation $R \equiv \{R_1, R_2, \ldots, R_n\}$ assuring no information loss, and avoiding certain non-controlled redundancies.
It is met iff $\bowtie_{i=1}^{n} \prod_{R_i}(R) = R$ (neither info loss, nor spurious tuples added).

<u>trivial JD</u>: when any relation in the decomposition is the original relation ($\exists\ R_i = R$)

5NF (aka projection-join normal form, or PJNF): R is in 5NF iff

- ✓ R is in 4NF and any decomposition of R with a **nontrivial JD** is implied by a superkey ($\cap R_i$ is a superkey)

<u>Example</u>:
Let be *supplying* (<u>branch, provider, product</u>) a relation verifying the assumption "any provider related to a branch is supplying all the products that provider supplies and that branch sells"; in this case, there is a decomposition of R:
supply_line(<u>branch,provider</u>), *supplier*(<u>provider,product</u>), *stocks*(<u>branch,product</u>), showing a nontrivial JD not implied by a superkey. Thus, that relation <u>is not in 5NF</u> **(but if it is substituted by the other three, the result will be in 5NF).**

Example of 5NF

branch	Provider	Product
1	X	hammer
1	X	nails
1	Y	screws
1	Y	nails
1	Z	cable
2	X	nails
2	Z	cable
3	Z	cable
1	Z	nails
2	Z	nails

note: branch 3 does not sell nails, so tuple (3,Z,nails) should not be added

Provider	Product
X	hammer
X	nails
Y	screws
Y	nails
Z	cable
Z	nails

branch	Provider
1	X
1	Y
1	Z
2	X
2	Z
3	Z

branch	Product
1	hammer
1	nails
1	screws
1	cable
2	nails
2	cable
3	cable

Advanced Normal Forms: DKNF

DKNF R is in DKNF iff all its restrictions are regarding either domain or key.

Example:
in the relation *pet*(id_pet, name, type, weight)
the attribute *type* is obtained by chaining the species with the term '_small' if the weight is below 1kg, or the term '_big' if it is greater than 2kg.

- Such *hidden semantic constraints* must be located and modelled in another way (with several single attributes, and in case, one or more derived attributes)

- In the example, the attribute 'species' should substitute *type*, and we could be adding another relation pet_type(species, min_weight, max_weight, tag), so we can derive the attribute *type* in the query (or by means of *views*).

Note: all 3NF, BCNF, 4NF y 5NF are particular cases of DKNF

Advanced Normal Forms: 6NF

6NF R is in 6NF iff it is in DKNF and it does not contain nontrivial JD.

- This is a **dangerous** NF, because it might be leading to a design *burst*. Taken to the extreme, it can result in a columnar decomposition of the tables.
- It is of great use, however, to separate attributes subject to temporal valuation (period of validity, category, contract, ...) from the attributes which are not.
- Often, relations coming from such decomposition are associated 1:1 to the *key table* (a table with the superkey, which may not exist), but space is saved and management of temporal attributes is simplified (at the cost of subsequent joins).
- It is especially useful in *history tables*, or bases of great historical content (datawarehouses)

Example:
Assignment (cod_emp, project_code, start_date, end_date)
It can be decomposed into *project_assign*(cod_emp, project_code, start_date), and *temp_assignment*(cod_emp, start_date, end_date)